Filters - GraphQL API

Overview

To control the amount of data returned by a query, items can be filtered.

This page explains the key technical concepts behind filtering in our API:

- how filter conditions are defined
- how multiple filters are combined
- common pitfalls to avoid

All examples use the filter available on the `searchServiceProviders` query. To show how filters are interpreted end-to-end, each GraphQL example is paired with the corresponding OData filter generated for Azure AI Search.

Example

Get all natural person service providers

```
searchServiceProviders(
  where: {
    isNaturalPerson: { eq: true }
) {
  totalCount
}
```

Filter structure

Different filters can be applied to limit the data returned. These filters can be combined using logical `AND` / `OR` operators.

ServiceProvider filters

For `ServiceProvider` search, the following main filters are available:

- `isNaturalPerson` : `true` / `false`
- `roles` : filter by roles
- `addresses` : filter by address attributes
- `features` : filter by feature attributes

Basic filtering

If multiple filters are defined at the root of the `where` clause, they must all match. In other words, they are combined using a logical `AND`.

Example

```
query {
  searchServiceProviders(
    searchTerm: "fribourg"
    where: {
      isNaturalPerson: { eq: false }
      roles: { some: { value: { eq: "Pharm" } } }
    }
  ) {
    totalCount
    nodes {
      name1
    }
  }
}
```

OData filter

```
(Roles/any(r: r/Value eq 'Pharm')) and (IsNaturalPerson eq false)
```

You can also express the same logic explicitly with an `and` array. The result is identical (apart from additional parentheses in the generated OData):

```
query {
  searchServiceProviders(
    searchTerm: "fribourg"
    where: {
      and: [
        { isNaturalPerson: { eq: false } }
        { roles: { some: { value: { eq: "Pharm" } } } }
      ]
    }
  ) {
    totalCount
    nodes {
      name1
    }
  }
}
```

OData filter

```
((IsNaturalPerson eq false) and (Roles/any(r: r/Value eq 'Pharm')))
```

Combining conditions with OR

When needed, conditions can be combined using the logical `OR` operator via the `or` array.

Example

```
query {
```

```
  searchServiceProviders(
    searchTerm: "fribourg"
    where: {
      or: [
        { isNaturalPerson: { eq: false } }
        { roles: { some: { value: { eq: "Pharm" } } } }
      ]
    }
  ) {
    totalCount
    nodes {
      name1
    }
  }
}
```

OData filter

```
((IsNaturalPerson eq false) or (Roles/any(r: r/Value eq 'Pharm')))
```

Important

Both conditions inside the `or` must be wrapped in their own curly braces (`{ ... }`). If the braces are omitted, the query may be syntactically valid, but the generated filter will **not** work as intended and will result in an `AND` combination of the filters.

Complex filters

Suppose we want to retrieve all pharmacies that:

1. are pharmacies (`roles` contains `"Pharm"`), **and**
2. either
   a. have a HIN email address, **or**
   b. are part of the Galenica group.

This can be expressed as:

```
query {
  searchServiceProviders(
    searchTerm: "fribourg"
    where: {
      # {1} ServiceProvider is a pharmacy
      roles: { some: { value: { eq: "Pharm" } } }
      or: [
        {
          # {2} Pharmacy has a HIN email address
          addresses: {
            some: {
              communications: {
                some: { communicationType: { value: { eq: "EmailHIN" } } }
              }
            }
          }
        }
        {
          # {3} Galenica pharmacy
```

```
          features: {
            some: {
              and: [
                { featureId: { eq: "PharmacyChainMember" } }
                { value: { in: ["Sunstore", "Amavita", "CoopVitality"] } }
              ]
            }
          }
        }
      ]
    }
  ) {
    totalCount
    nodes {
      name1
    }
  }
}
```

OData filter

```
((Addresses/any(a: a/Communications/any(c: c/CommunicationType/Value eq 'EmailHIN')))
 or (Features/any(f: ((f/FeatureId eq 'PharmacyChainMember')
   and (f/Value eq 'Sunstore' or f/Value eq 'Amavita' or f/Value eq 'CoopVitality')))))
and (Roles/any(r: r/Value eq 'Pharm'))
```

Note the logical structure:

```
({2} OR {3}) AND {1}
```

Common pitfalls

1. All `OR` conditions must be inside the `or` array

When using the `or` operator, **every** condition that should be part of the `OR` must be placed inside the `or: [ ... ]` array, and each must be wrapped in `{ ... }`.

✅ Correct

```
where: {
  or: [
    { isNaturalPerson: { eq: false } }
    { roles: { some: { value: { eq: "Pharm" } } } }
  ]
}
```

OData filter

```
((IsNaturalPerson eq false) or (Roles/any(r: r/Value eq 'Pharm')))
```

❌ Incorrect – missing braces around entries

```
where: {
  or: [
    isNaturalPerson: { eq: false }
    roles: { some: { value: { eq: "Pharm" } } }
  ]
```

```
}
```

OData filter

```
((Roles/any(r: r/Value eq 'Pharm'))) and (IsNaturalPerson eq false)
```

The filter unintentionally becomes an `AND` combination.

❌ Incorrect – mixing root condition with `or`

```
where: {
  isNaturalPerson: { eq: false }
  or: [
    { roles: { some: { value: { eq: "Pharm" } } } }
  ]
}
```

OData filter

```
((Roles/any(r: r/Value eq 'Pharm'))) and (IsNaturalPerson eq false)
```

Here, `isNaturalPerson` is always `AND`-ed with whatever is inside `or`, which may not be the intended logic.